

Using the Recursive Backtracker Algorithm

The `visit()` function implements the recursive backtracker algorithm. The function has a list (in Python) or array (in JavaScript) that keeps track of the `x`, `y` coordinates that have already been visited by previous `visit()` function calls. It also in-place modifies the global `maze` variable that stores the maze data structure. The Python code for `visit()` begins as follows:

Python

```
def visit(x, y):
    """Carve out" empty spaces in the maze at x, y and then
    recursively move to neighboring unvisited spaces. This
    function backtracks when the mark has reached a dead end."""
    maze[(x, y)] = EMPTY # "Carve out" the space at x, y.
    printMaze(maze, x, y) # Display the maze as we generate it.
    print('\n\n')
```

The JavaScript code for `visit()` begins as follows:

JavaScript

```
function visit(x, y) {
    // "Carve out" empty spaces in the maze at x, y and then
    // recursively move to neighboring unvisited spaces. This
    // function backtracks when the mark has reached a dead end.

    maze[[x, y]] = EMPTY; // "Carve out" the space at x, y.
    printMaze(maze, x, y); // Display the maze as we generate it.
    document.write('<br /><br /><br />');
```

The `visit()` function accepts `x`, `y` coordinates as arguments for the place in the maze the algorithm is visiting. Then the function changes the data structure in `maze` at this location to an empty space. To let the user see the progression of the maze generation, it calls `printMaze()`, passing the `x` and `y` arguments as the current position of the mark.

Next, the recursive backtracker calls `visit()` with the coordinates of a previously unvisited neighboring space. The Python code continues as follows: